FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Eric V. Tividad Jr.
Section:	AN 42
Professor:	Ms. Crisola Tan
Date:	10/2/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import Counter

animations = []

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def setup(self):
        self.path = [] # stores positions of the dots

    def step(self):
        p = self.model.p
        dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)] # dot movement

        # for task 2: add more movement to the right
        if p.biased:
            dirs += [(1, 0)] * 3
        random.shuffle(dirs)

        # gets dots current spot, and tracks which location has been taken
        x, y = self.position
        taken = {a.position for a in self.model.agents if a is not self}

        for dx, dy in dirs:
            nx = max(0, min(p.grid_size[0] - 1, x + dx))
            ny = max(0, min(p.grid_size[1] - 1, y + dy))

            # skip cells if occupied
            if p.avoid and (nx, ny) in taken:
```

```
                continue
            self.position = (nx, ny)
            break
        self.path.append(self.position)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # i create ze agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # put agents on random starting location
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))
            agent.path.append(agent.position) # starting point

    # let agents gets their own turn
    def step(self):
        for agent in self.agents:
            agent.step()

def run(agents, size, steps, avoid=False, biased=False, label="Run"):
    params = {'agents': agents,
              'grid_size': (size, size),
              'steps': steps,
              'avoid': avoid,
              'biased': biased}

    model = RandomWalkModel(params)
    model.setup()

    for _ in range(steps):
        model.step()

    # for task 3: collects final position
    print(label)
    print(f"Agents={agents}, Grid={size}x{size}, Steps={steps}, avoid={avoid}, biased={biased}")
    finals = [a.position for a in model.agents]
    for i, pos in enumerate(finals, 1):
        print(f"  Agent {i}: final position = {pos}")
    avg_x = sum(p[0] for p in finals) / len(finals)
    avg_y = sum(p[1] for p in finals) / len(finals)
    print(f"  Average final position: ({avg_x:.2f}, {avg_y:.2f})")
```

```
    print(f"  Cells with 2+ agents: {sum(1 for c in Counter(finals).values() if c > 1)}")

    # for task 1: plotting paths
    fig, ax = plt.subplots()
    for i, a in enumerate(model.agents, 1):
        xs = [p[0] for p in a.path]; ys = [p[1] for p in a.path]
        ax.plot(xs, ys, marker='.', alpha=0.6, label=f"Agent {i}")
        ax.scatter(xs[0], ys[0], marker='s', s=60)     # start
        ax.scatter(xs[-1], ys[-1], marker='*', s=150)  # end

    ax.set_xlim(-0.5, size - 0.5); ax.set_ylim(-0.5, size - 0.5)

    ax.set_xticks(range(size)); ax.set_yticks(range(size)); ax.grid(True)

    ax.set_title(f"{label}: paths (square=start, star=end)")

    if agents <= 10:
        ax.legend(fontsize=7)

    # -------- Animation -----------
    # replays the saved paths of the agents, one step per frame
    fig2, ax2 = plt.subplots()
    ax2.set_xlim(-0.5, size - 0.5); ax2.set_ylim(-0.5, size - 0.5)
    ax2.set_xticks(range(size)); ax2.set_yticks(range(size)); ax2.grid(True)
    scat = ax2.scatter([], [], s=200) # s = size of agents

    def update(frame):
        # frame 0 = starting positions, frame 1 = after step 1, and so on
        x = [a.path[frame][0] for a in model.agents]
        y = [a.path[frame][1] for a in model.agents]
        scat.set_offsets(list(zip(x, y)))
        ax2.set_title(f"{label}: step {frame}/{steps}")
        return scat,

    ani = animation.FuncAnimation(fig2, update, frames=steps + 1,
                                  blit=False, repeat=False)
    animations.append(ani) # keep a reference so the animation doesn't get deleted

    return finals

print("Technical Assesment 1: Random Walk Simulation Using AgentPY")
print("1 - Scenario A (5 agents, 10x10 grid, 20 steps, plain random walk)")
print("2 - Scenario B (20 agents, 10x10 grid, 50 steps, avoid + preferred direction)")
print("3 - Enter my own values")
```

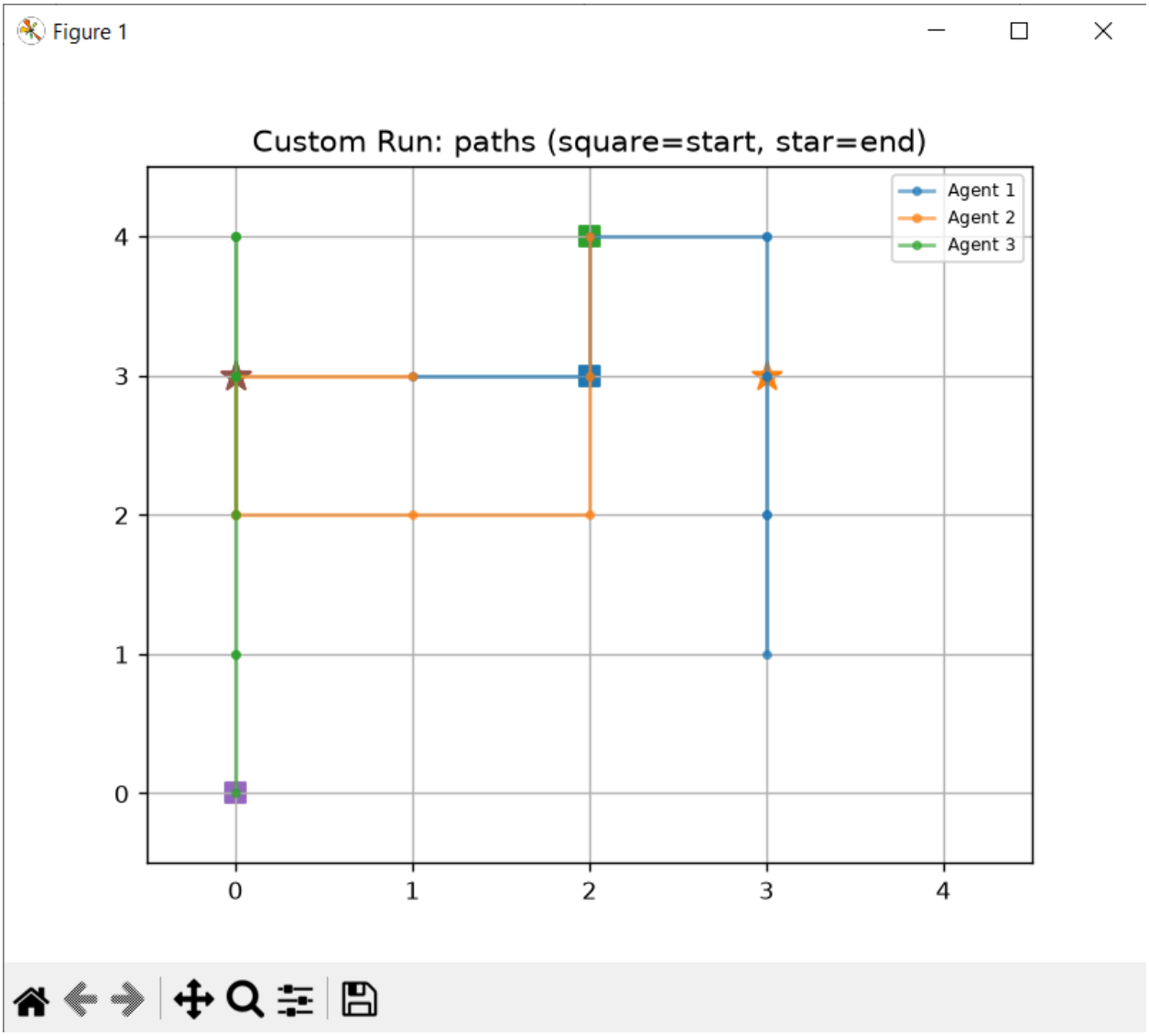
```
choice = input("Pick a mode: ").strip()

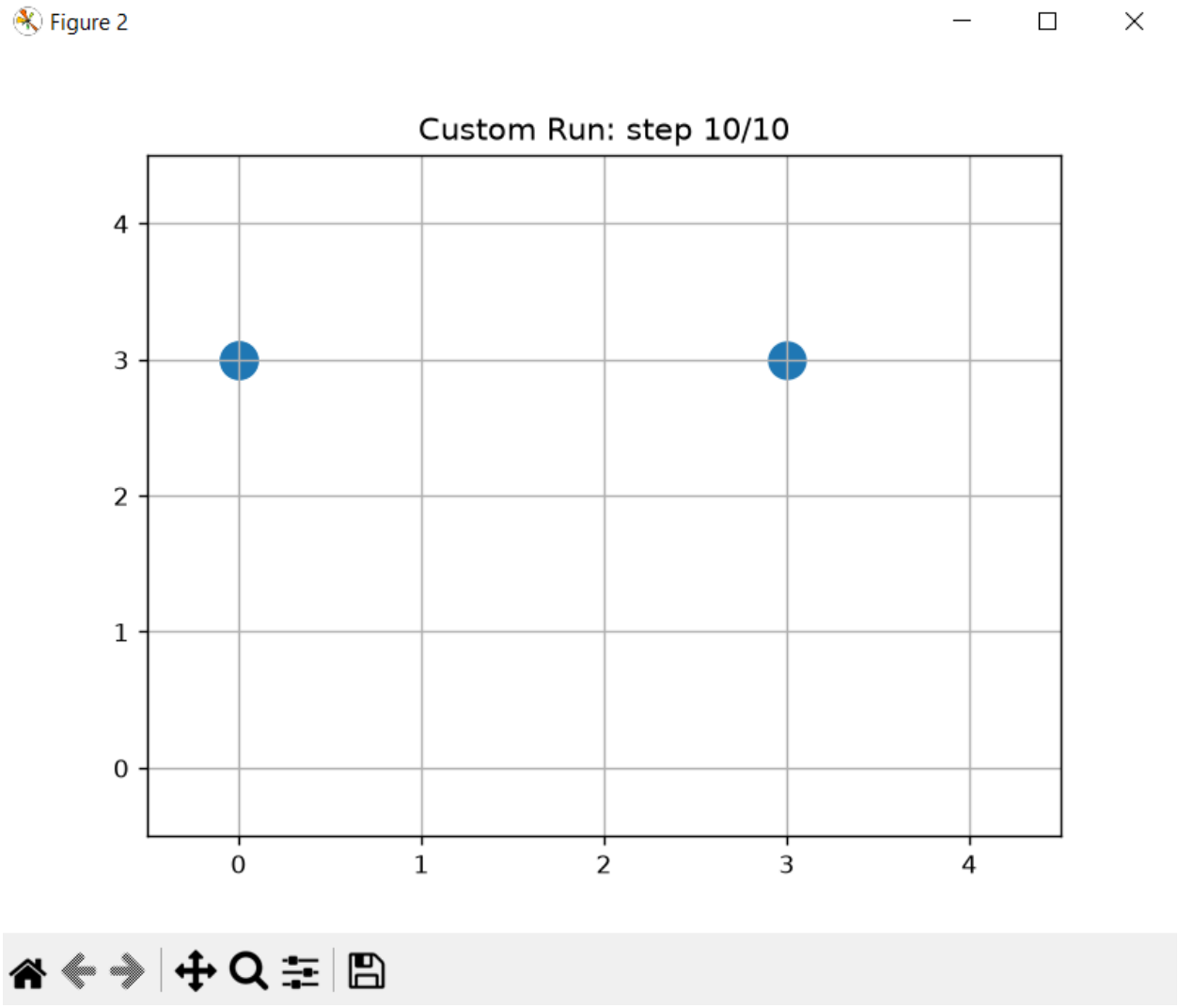
if choice == "1":
    run(5, 10, 20, label="Scenario A")
elif choice == "2":
    run(20, 10, 50, avoid=True, biased=True, label="Scenario B")
elif choice == "3":
    n_agents = int(input("Number of agents: "))
    grid = int(input("Grid size (e.g., 10 for 10x10): "))
    n_steps = int(input("Number of steps: "))
    run(n_agents, grid, n_steps, label="Custom Run")
else:
    print("Invalid choice. Please run again and enter 1, 2, or 3.")
    exit()

plt.show()
```

Output – Custom Run (Vanilla):

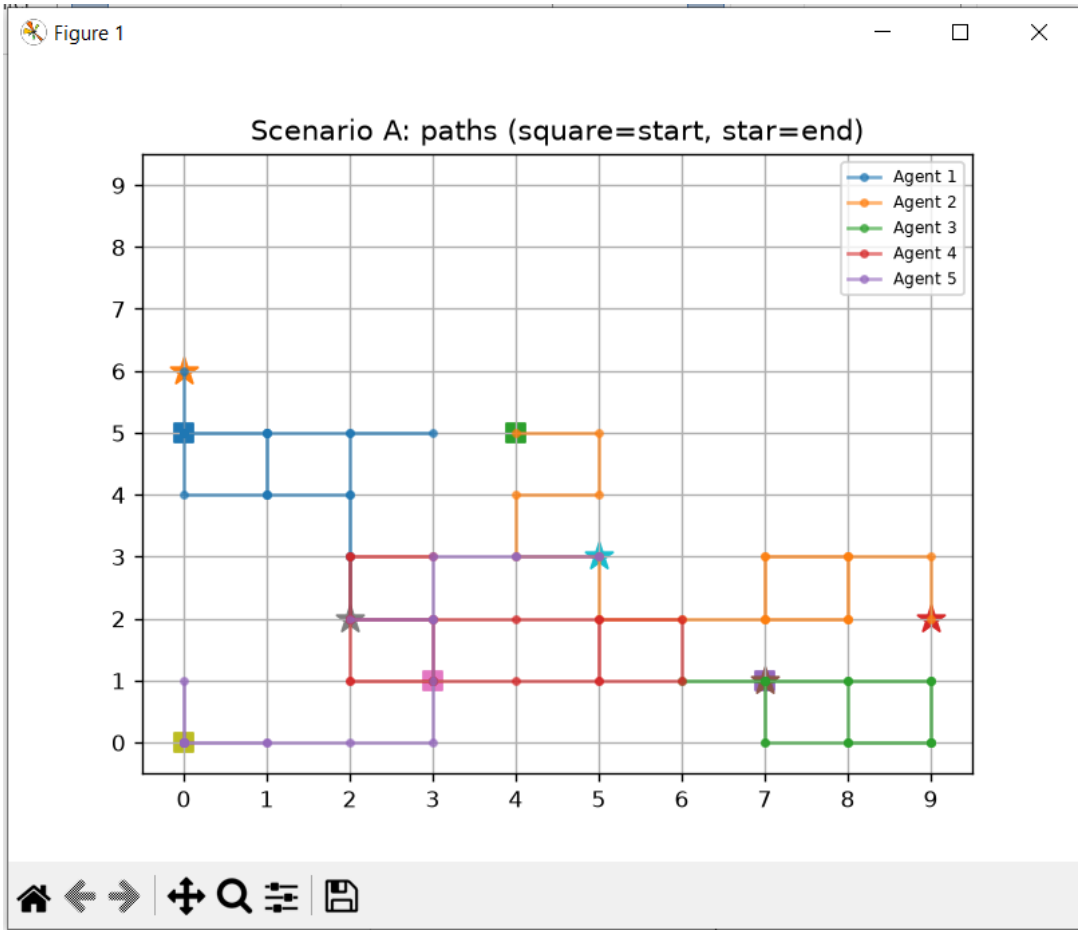
```
Technical Assesment 1: Random Walk Simulation Using AgentPY
1 - Scenario A (5 agents, 10x10 grid, 20 steps, plain random walk)
2 - Scenario B (20 agents, 10x10 grid, 50 steps, avoid + preferred direction)
3 - Enter my own values
Pick a mode: 3
Number of agents: 3
Grid size (e.g., 10 for 10x10): 5
Number of steps: 10
Custom Run
Agents=3, Grid=5x5, Steps=10, avoid=False, biased=False
  Agent 1: final position = (3, 3)
  Agent 2: final position = (0, 3)
  Agent 3: final position = (0, 3)
  Average final position: (1.00, 3.00)
  Cells with 2+ agents: 1
```

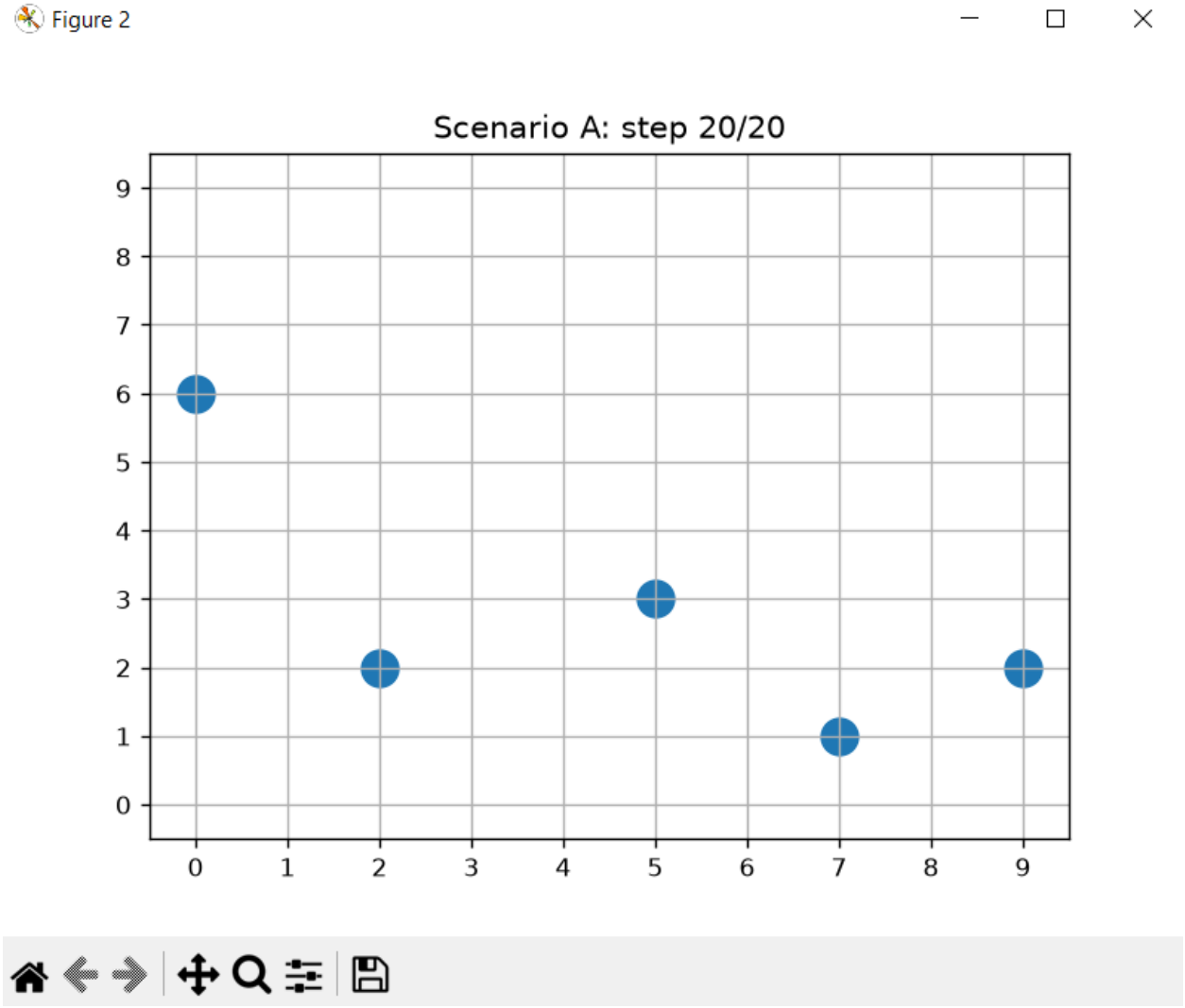

Figure 1
Custom Run: paths (square=start, star=end)
Agent 1
Agent 2
Agent 3
0
1
2
3
4
0
1
2
3
4


Figure 2
Custom Run: step 10/10
4
3
2
1
0
0
1
2
3
4

Output – Scenario A Run:

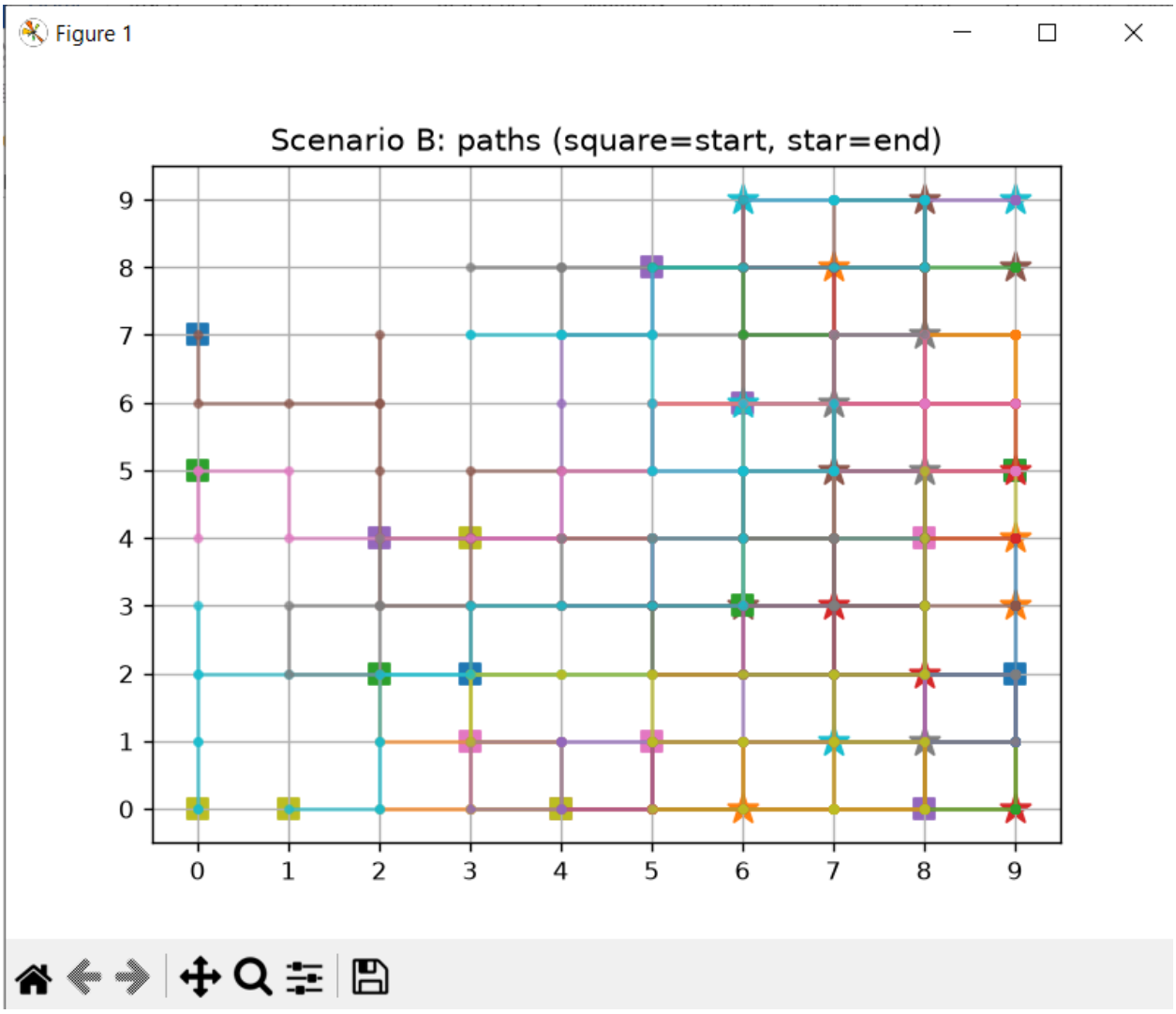
```
Technical Assesment 1: Random Walk Simulation Using AgentPY
1 - Scenario A (5 agents, 10x10 grid, 20 steps, plain random walk)
2 - Scenario B (20 agents, 10x10 grid, 50 steps, avoid + preferred direction)
3 - Enter my own values
Pick a mode: 1
Scenario A
Agents=5, Grid=10x10, Steps=20, avoid=False, biased=False
  Agent 1: final position = (0, 6)
  Agent 2: final position = (9, 2)
  Agent 3: final position = (7, 1)
  Agent 4: final position = (2, 2)
  Agent 5: final position = (5, 3)
  Average final position: (4.60, 2.80)
  Cells with 2+ agents: 0
```

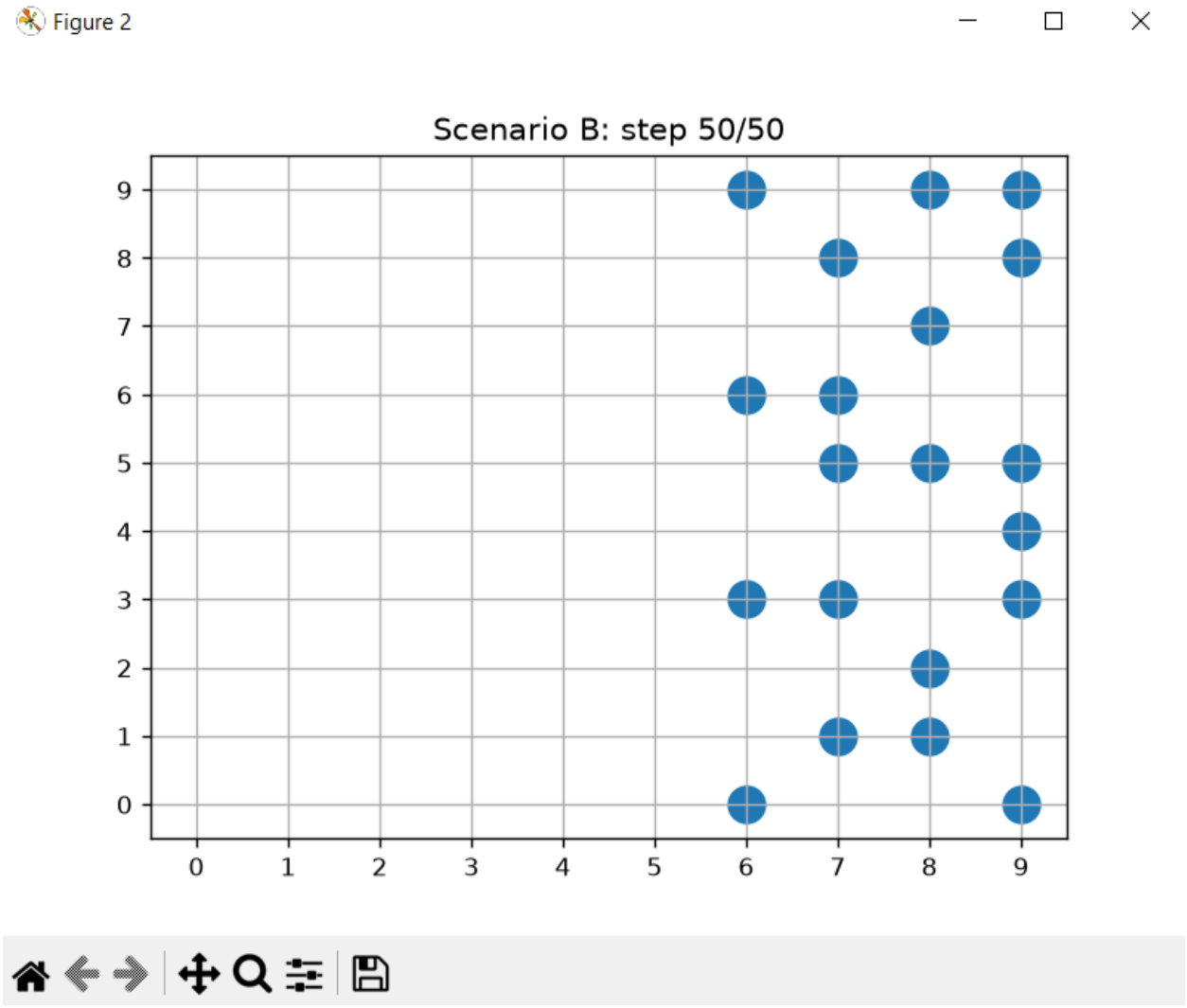



Figure 2
Scenario A: step 20/20
9
8
7
6
5
4
3
2
1
0
0
1
2
3
4
5
6
7
8
9

Output – Scenario B Run:

```
Technical Assesment 1: Random Walk Simulation Using AgentPY
1 - Scenario A (5 agents, 10x10 grid, 20 steps, plain random walk)
2 - Scenario B (20 agents, 10x10 grid, 50 steps, avoid + preferred direction)
3 - Enter my own values
Pick a mode: 2
Scenario B
Agents=20, Grid=10x10, Steps=50, avoid=True, biased=True
  Agent 1: final position = (6, 0)
  Agent 2: final position = (9, 0)
  Agent 3: final position = (6, 3)
  Agent 4: final position = (8, 1)
  Agent 5: final position = (9, 9)
  Agent 6: final position = (7, 8)
  Agent 7: final position = (8, 2)
  Agent 8: final position = (7, 5)
  Agent 9: final position = (8, 7)
  Agent 10: final position = (6, 6)
  Agent 11: final position = (9, 4)
  Agent 12: final position = (7, 3)
  Agent 13: final position = (9, 8)
  Agent 14: final position = (7, 6)
  Agent 15: final position = (7, 1)
  Agent 16: final position = (9, 3)
  Agent 17: final position = (9, 5)
  Agent 18: final position = (8, 9)
  Agent 19: final position = (8, 5)
  Agent 20: final position = (6, 9)
  Average final position: (7.65, 4.70)
  Cells with 2+ agents: 0
```


Figure 1
Scenario B: paths (square=start, star=end)
0
1
2
3
4
5
6
7
8
9



Answer the Questions ⇩
Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
   - The agents move the same, but they tend to look more cramped when executing it. For example, with an agent of 10 on a 5x5 grid, it can be observed that a few sections will have less movement due to agents staying relative around that area, unlike on an agent of 3 on a 10x10 grid.

2. How does increasing the number of steps affect the **spread of agents**?
   - The higher the steps, the wider the spread of the agents. This because higher steps create more pool for the agents to actually move on more areas.

3. Do the agents ever move outside the grid? Why or why not?
    - No, the agents don't ever move outside the grid. The code is written that it can only move around the grid, with its coordinate and all.

4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
    - Simulation like these can be used on movement-based studies. Examples of such are crowd-management, traffics, animal behavior, or even tracking why and how could a disease spread.